

Mathematics for Machine Learning — Part 3

Placement-focused notes on optimization, information theory, and Bayesian AI. Examples use Python 3 and NumPy.

Lagrange Multipliers

1. Overview

Lagrange multipliers optimize an objective under constraints. They are used in SVM duality, constrained regression, and resource allocation.

2. Intuition

At a constrained optimum, the objective cannot improve along the allowed surface, so its gradient is parallel to the constraint gradient.

3. Prerequisites

Partial derivatives, gradients, linear algebra, and optimization.

4. Core Concepts

- **Constraint:** $g(x)=0$ defines feasible points; interview angle: distinguish it from $g(x)\leq 0$.
- **Lagrangian:** $L=f+\lambda g$ combines objective and constraint.
- **Stationarity:** $\nabla f+\lambda \nabla g=0$; λ is a shadow price.
- **KKT:** extends the method to inequalities; key for SVMs.

5. Algorithm / Working Process

1. Write f and constraints.
2. Form $L=f+\sum_i \lambda_i g_i$.
3. Set parameter and multiplier derivatives to zero.
4. Keep feasible candidates and compare values.

6. Mathematical Foundation

For $(\min f(x))$ subject to $g(x)=0$, solve $\nabla_x L=0$ and $g(x)=0$. KKT adds primal feasibility, $\lambda \geq 0$, and $\lambda g(x)=0$ for inequalities.

7. Practical Implementation

```
import numpy as np
x = y = 1 / np.sqrt(2) # max x+y, subject to x^2+y^2=1
lam = -1 / (2 * x)
```

```
assert np.isclose(x*x + y*y, 1)
print(x + y, lam)
```

8. Code Explanation

Stationarity gives $(x=y)$; substitution into the constraint gives $(1/\sqrt{2})$. The assertion checks feasibility.

9. Training / Evaluation

Check objective, constraint violation, and KKT residuals. Tune constraint budgets or regularization on validation data.

10. Complexity and Cost

Closed forms are cheap; general smooth solvers iterate over gradients and can require dense ($O(d^3)$) solves.

11. Common Use Cases

SVMs, norm constraints, portfolio limits, maximum entropy, and differentiable physics.

12. Common Mistakes

Omitting feasibility, reversing inequality signs, assuming stationarity is global optimality, and poor scaling.

13. Edge Cases / Limitations

Non-convex problems have local optima; degenerate or inconsistent constraints break assumptions.

14. Variations

- **Multiple constraints:** one multiplier each; placement essential.
- **KKT conditions:** inequality generalization; essential for SVM interviews.
- **Augmented Lagrangian:** penalty-enhanced numerical method; research/practical.

15. Related Topics

Penalty methods approximate constraints; duality reformulates them; gradient descent handles unconstrained objectives.

16. Interview Questions

1. **What is (λ) ?** Sensitivity to relaxing a constraint.
2. **Why parallel gradients?** Both are normals to the feasible surface.
3. **Does stationarity guarantee a maximum?** No.
4. **Many constraints?** Add one multiplier per constraint.
5. **Inequalities?** Use KKT.
6. **Complementary slackness?** $(\lambda g(x)=0)$.
7. **SVM connection?** It produces the dual and support-vector conditions.
8. **Can equality (λ) be negative?** Yes.

9. **Penalty versus exact constraint?** Penalties may violate feasibility.

10. **When can KKT fail?** Failed constraint qualification.

17. Practice Tasks

Maximize a product with fixed sum; solve two constraints; derive a norm-constrained regression form; compare penalty and KKT solutions.

18. Project Ideas

- **Constrained portfolio:** NumPy/SciPy and market returns; optimization signal.
- **SVM margin visualizer:** scikit-learn and synthetic blobs; interview-friendly.
- **Fair allocation solver:** SciPy and demand/cap data; operations-ML value.

19. Quick Revision

Key idea: optimize only on an allowed surface. Formula: $(\nabla f + \lambda \nabla g = 0)$. Trap: missing feasibility. One-liner: (λ) is a constraint's shadow price.

20. Final Cheat Sheet

Item	Summary
Definition	Constrained optimization
Input/output	objective + constraints $\rightarrow$ feasible optimum
Steps	Lagrangian, derivatives, solve
Metrics	objective and violation
Pros/cons	principled / numerically hard
Best uses	SVMs and budgets

KL Divergence

1. Overview

KL divergence measures directed mismatch between distributions. It powers VAEs, variational inference, distillation, and drift monitoring.

2. Intuition

It is the extra average coding cost from using (Q) when data are produced by (P) .

3. Prerequisites

Probability, expectations, logarithms, entropy, and calculus.

4. Core Concepts

- **Definition:** $D_{\text{KL}}(P||Q) = -\sum P \log(P/Q)$; use an integral for densities.
- **Asymmetry:** $\text{KL}(P||Q) \neq \text{KL}(Q||P)$; direction changes behavior.
- **Non-negativity:** it is at least zero, but not a metric.
- **Support:** target mass where model mass is zero gives infinite forward KL.

5. Algorithm / Working Process

1. Normalize target and model probabilities.
2. Compute the log ratio safely.
3. Weight it by target probability.
4. Sum and minimize when fitting the model.

6. Mathematical Foundation

$\text{KL}(P||Q) = H(P, Q) - H(P)$. Thus minimizing KL to fixed (P) minimizes cross entropy. Gaussian VAE KL is $\frac{1}{2}(\sum(\mu^2 + \sigma^2 - \log \sigma^2 - 1))$.

7. Practical Implementation

```
import numpy as np
p, q = np.array([.7, .2, .1]), np.array([.6, .3, .1])
kl = np.sum(p * np.log(p / q))
assert kl >= -1e-12
print(kl)
```

8. Code Explanation

The vectors are normalized PMFs. The formula is computed elementwise; tolerance handles floating-point error.

9. Training / Evaluation

For generative models, assess held-out likelihood/ELBO, samples, and calibration too. High-dimensional sample KL estimates are difficult.

10. Complexity and Cost

Discrete KL is $O(k)$. Neural KL objectives cost a forward/backward pass; Monte Carlo estimates add variance.

11. Common Use Cases

VAE latent regularization, distillation, PPO penalties, language-model comparison, and data-drift alerts.

12. Common Mistakes

Reversing arguments, treating KL as symmetric, using logits as probabilities, and ignoring zero support.

13. Edge Cases / Limitations

It may be infinite, is rare-event sensitive, and one number does not localize the mismatch.

14. Variations

- **Forward/reverse KL:** mode-covering versus mode-seeking; research important.
- **Jensen–Shannon:** symmetric, bounded; GAN-related.
- **KL annealing:** raises VAE KL weight gradually; reduces collapse.

15. Related Topics

Cross entropy equals entropy plus KL; ELBO contains KL; Wasserstein and total variation are alternatives.

16. Interview Questions

1. **What is KL?** Expected log-ratio mismatch.
2. **Metric?** No.
3. **When zero?** Equal almost everywhere.
4. **Why non-negative?** Gibbs' inequality.
5. **When infinite?** ($Q=0$) where ($P>0$).
6. **VI direction?** Usually ($KL(q||p)$).
7. **CE relation?** ($CE=H+KL$).
8. **Why logs?** Stability and additive information.
9. **Why mode dropping?** Reverse KL avoids low-target-density regions.
10. **VAE role?** Pulls posterior approximation toward prior.

17. Practice Tasks

Compute both directions; plot Bernoulli KL; implement Gaussian KL; compare teacher/student outputs.

18. Project Ideas

- **Drift monitor:** NumPy/Pandas prediction distributions; MLOps value.
- **Distillation study:** PyTorch/CIFAR-10; deep-learning value.
- **VAE dashboard:** PyTorch/MNIST; generative-AI value.

19. Quick Revision

Key idea: directed mismatch. Formula: ($E_P[\log(P/Q)]$). Trap: order matters. One-liner: fixed-target KL minimization is CE minimization.

20. Final Cheat Sheet

Item	Summary
Definition	Directed divergence
Input/output	$P, Q \rightarrow$ mismatch
Steps	normalize, log ratio, expectation
Metrics	KL, ELBO, likelihood

Item	Summary
Pros/cons	principled / asymmetric and may be infinite
Best uses	VAEs, VI, distillation

Entropy

1. Overview

Entropy measures uncertainty in a distribution. ML uses it for decision-tree splits, active learning, compression, and confidence diagnostics.

2. Intuition

A fair coin is harder to predict than one that always lands heads; entropy is average surprise before observation.

3. Prerequisites

Probability, logarithms, expectations, and classification probabilities.

4. Core Concepts

- **Self-information:** $I(x) = -\log p(x)$; rare outcomes surprise more.
- **Shannon entropy:** $H(X) = -\sum p \log p$; expected surprise.
- **Maximum:** uniform (k)-class probabilities give $(\log k)$.
- **Conditional entropy:** $H(Y|X)$ is uncertainty remaining after (X).

5. Algorithm / Working Process

1. Count categories or obtain probabilities.
2. Normalize.
3. Compute $(-p \log p)$, defining $(0 \log 0 = 0)$.
4. Sum and compare distributions.

6. Mathematical Foundation

Base 2 gives bits; natural logs give nats. Independent variables satisfy $H(X,Y) = H(X) + H(Y)$, and $H(Y|X) \leq H(Y)$.

7. Practical Implementation

```
import numpy as np
p = np.array([.5, .25, .25])
h = -np.sum(p * np.log2(p))
assert np.isclose(h, 1.5)
print(h)
```

8. Code Explanation

The PMF is normalized; log base 2 reports bits. Its weighted surprises total 1.5.

9. Training / Evaluation

Use entropy with accuracy, calibration, AUROC, and OOD checks; it is a diagnostic, not a replacement for task metrics.

10. Complexity and Cost

($O(k)$) time/memory for (k) categories; no training or GPU.

11. Common Use Cases

Tree splits, active-learning selection, token uncertainty, compression, and probabilistic debugging.

12. Common Mistakes

Raw counts, mixed log bases, calling uncertainty error, and taking log of zero directly.

13. Edge Cases / Limitations

High entropy can be appropriate ambiguity; it ignores error severity. Differential entropy depends on units.

14. Variations

- **Joint/conditional entropy:** shared/remaining uncertainty; placement relevant.
- **Differential entropy:** continuous variables; statistics.
- **Rényi entropy:** tail-sensitive extension; research.

15. Related Topics

Information gain is entropy reduction; cross entropy compares target/prediction; KL is cross entropy minus target entropy.

16. Interview Questions

1. **Definition?** Expected surprise.
2. **Maximum when?** Uniform distribution.
3. **Deterministic entropy?** Zero.
4. **Why log?** Independent information adds.
5. **Bits/nats?** Base 2/base (e).
6. **High entropy wrong?** No.
7. **Conditional entropy?** Remaining uncertainty.
8. **Zero probability term?** Zero.
9. **Tree use?** Choose entropy-reducing splits.
10. **Versus variance?** Entropy covers distributions/categories.

17. Practice Tasks

Plot Bernoulli entropy; estimate it from labels; compare classifier confidence; implement conditional entropy.

18. Project Ideas

- **Active learner:** PyTorch/CIFAR-10; picks high-entropy samples.
- **LLM uncertainty viewer:** Transformers; displays token entropy.
- **Tree-split explorer:** scikit-learn/Titanic; explains information reduction.

19. Quick Revision

Key idea: average uncertainty. Formula: $(-\sum p \log p)$. Trap: unnormalized probabilities. One-liner: uniform outcomes maximize entropy.

20. Final Cheat Sheet

Item	Summary
Definition	Average surprise
Input/output	probabilities $\rightarrow$ bits/nats
Steps	normalize, $(-p \log p)$, sum
Metrics	entropy plus calibration
Pros/cons	interpretable / not correctness
Best uses	trees, active learning

Cross Entropy

1. Overview

Cross entropy scores predicted probabilities against true outcomes. It is the default loss for classifiers, language models, and segmentation.

2. Intuition

It heavily penalizes assigning tiny probability to what actually happened.

3. Prerequisites

Probability, entropy, KL, sigmoid, softmax, and gradients.

4. Core Concepts

- **Multiclass CE:** $H(P, Q) = -\sum P \log Q$; one-hot labels give $(-\log q_y)$.
- **Binary CE:** $(-[y \log p + (1-y) \log (1-p)])$; use for binary/multilabel data.
- **Logits:** give raw logits to stable loss APIs; do not apply softmax/sigmoid twice.
- **Gradient:** softmax-CE gradient is predicted probability minus one-hot target.

5. Algorithm / Working Process

1. Model outputs logits.
2. Conceptually apply softmax/sigmoid.
3. Compute negative log likelihood.
4. Backpropagate and update weights.

6. Mathematical Foundation

$H(P,Q)=H(P)+KL(P||Q)$, so minimizing CE for fixed labels minimizes KL. Softmax is $(e^{z_i}/\sum_j e^{z_j})$.

7. Practical Implementation

```
import numpy as np
logits, target = np.array([1.2, -.3, 2.0]), 2
log_probs = logits - np.log(np.exp(logits).sum())
loss = -log_probs[target]
assert loss > 0
print(loss)
```

8. Code Explanation

Log-probabilities are formed, then indexing the true class implements one-hot CE without materializing a target vector.

9. Training / Evaluation

Use stratified splits; monitor CE plus accuracy, F1, AUROC, IoU, or perplexity. Tune learning rate, weights, label smoothing, and stopping point.

10. Complexity and Cost

Dense softmax is $O(k)$ per example; vocabulary softmax dominates many LLMs. GPU need depends on model size.

11. Common Use Cases

Image classification, next-token prediction, segmentation, spam detection, and multilabel tagging.

12. Common Mistakes

Double softmax, wrong label shape/dtype, BCE for exclusive classes, ignoring imbalance, and reporting only accuracy.

13. Edge Cases / Limitations

CE does not directly optimize F1; label noise encourages overconfidence; extreme logits need stable implementations.

14. Variations

- **Weighted CE:** makes rare classes matter; project essential.
- **Label smoothing:** softens targets; reduces overconfidence.
- **Focal loss:** emphasizes hard examples; detection use.

15. Related Topics

Negative log likelihood is equivalent for categorical labels; KL drives distillation; perplexity is exponentiated token CE.

16. Interview Questions

1. **Why CE?** It is negative log likelihood.
2. **Perfect correct prediction?** Loss approaches zero.
3. **BCE vs CE?** Independent labels versus one exclusive class.
4. **Why logits?** Stable fused implementation.
5. **KL relation?** CE equals entropy plus KL.
6. **Label smoothing?** Slightly soft targets.
7. **Perplexity?** Exponential average token CE.
8. **Negative CE?** Not for valid standard probabilities.
9. **Imbalance fix?** Weights, sampling, focal loss.
10. **Accuracy up but CE down?** Calibration/confidence can worsen.

17. Practice Tasks

Implement BCE; compare confident correct/wrong loss; train MNIST; add class weights; plot calibration.

18. Project Ideas

- **Spam classifier:** TF-IDF/logistic regression, SMS Spam; classical ML.
- **CIFAR classifier:** PyTorch; compare CE and focal loss.
- **Tiny language model:** PyTorch/text corpus; report perplexity.

19. Quick Revision

Key idea: punish low probability on truth. Formula: $(-\sum y \log p)$. Trap: double softmax. One-liner: CE is maximum likelihood as a loss.

20. Final Cheat Sheet

Item	Summary
Definition	Target-weighted negative log probability
Input/output	logits + labels $\rightarrow$ loss
Steps	normalize, NLL, backprop
Metrics	CE, accuracy/F1/AUROC/perplexity

Item	Summary
Pros/cons	strong gradients / label-noise sensitive
Best uses	classifiers, LMs

Information Gain

1. Overview

Information gain (IG) measures how much a feature reduces target uncertainty. It is the classic ID3 decision-tree split criterion and a mutual-information feature score.

2. Intuition

A question is useful when its answer makes the label easier to predict. A useful split creates purer child groups.

3. Prerequisites

Entropy, conditional probability, categorical features, decision trees, and weighted averages.

4. Core Concepts

- **Formula:** $(IG(Y;X)=H(Y)-H(Y|X))$; uncertainty removed by (X).
- **Child weighting:** child entropy is weighted by child size.
- **Mutual information:** IG is mutual information between feature and target.
- **Cardinality bias:** IDs can receive high IG; gain ratio helps.

5. Algorithm / Working Process

1. Compute parent-label entropy.
2. Partition data for each candidate split.
3. Compute weighted child entropy.
4. Select greatest gain and recurse with stopping rules.

6. Mathematical Foundation

$(H(Y|X)=\sum_x P(x)H(Y|X=x))$. Therefore $(IG=H(Y)-\sum_x P(x)H(Y|x))$. Pure children have zero entropy; numeric features use thresholds.

7. Practical Implementation

```
import numpy as np
def entropy(y):
    _, c = np.unique(y, return_counts=True); p = c / c.sum()
    return -np.sum(p * np.log2(p))
y = np.array([0, 0, 1, 1]); groups = [y[:2], y[2:]]
```

```
ig = entropy(y) - sum(len(g)/len(y)*entropy(g) for g in groups)
assert np.isclose(ig, 1); print(ig)
```

8. Code Explanation

The parent has one bit of uncertainty and both children are pure, so the split gains one bit.

9. Training / Evaluation

Fit splits on training data only; tune max depth and minimum samples using validation/cross-validation. Report accuracy, F1, and tree stability.

10. Complexity and Cost

Optimized trees approximately sort numeric features in $(O(nd \log n))$ per level. Inference follows one path, $(O(\text{depth}))$.

11. Common Use Cases

ID3 trees, feature selection, explainable rules, and discrete Bayesian networks.

12. Common Mistakes

Leaking ID-like features, not weighting children, target leakage, and unlimited depth.

13. Edge Cases / Limitations

IG favors high-cardinality features, is unstable on small data, and greedy splits can miss the global tree.

14. Variations

- **Gain ratio:** normalizes split information; C4.5.
- **Gini decrease:** cheaper CART alternative; common in random forests.
- **Mutual-information selection:** ranks features before another model.

15. Related Topics

Entropy is baseline uncertainty; conditional entropy is remainder; random forests average greedy trees.

16. Interview Questions

1. **Define IG.** Parent entropy minus conditional entropy.
2. **High IG means?** More predictable labels after split.
3. **Why weight children?** Large groups must matter more.
4. **Pure-parent IG?** Zero.
5. **Main bias?** High-cardinality features.
6. **Fix?** Gain ratio/constraints.
7. **IG vs Gini?** Both purity scores; Gini avoids logs.
8. **Can population IG be negative?** No.

- 9. **Numeric split?** Search thresholds.
- 10. **Why overfit?** Tiny partitions can be pure.

17. Practice Tasks

Compute a split by hand; implement threshold search; compare IG/Gini; demonstrate ID leakage.

18. Project Ideas

- **Loan approval tree:** scikit-learn/German Credit; interpretable ML.
- **Feature-ranking audit:** Pandas/scikit-learn/UCI data; feature-engineering value.
- **Medical triage rules:** heart-disease data with fairness checks; responsible-AI value.

19. Quick Revision

Key idea: uncertainty removed by a feature. Formula: $(H(Y) - H(Y|X))$. Trap: cardinality bias. One-liner: IG is mutual information for a split.

20. Final Cheat Sheet

Item	Summary
Definition	Entropy reduction from a split
Input/output	labels + feature $\rightarrow$ score
Steps	parent minus weighted children
Metrics	IG, depth, validation F1
Pros/cons	interpretable / cardinality bias
Best uses	ID3 trees, feature selection

Measure Theory Basics

1. Overview

Measure theory rigorously defines probability, integration, and almost-sure claims. It underlies continuous distributions, stochastic processes, and Bayesian probability.

2. Intuition

Length gives intervals a size; a measure generalizes size to sets. Probability is a measure whose total mass is one, while a sigma-algebra lists allowed events.

3. Prerequisites

Sets, limits, basic probability, integrals, and proof comfort.

4. Core Concepts

- **Measurable space** $(\Omega, \mathcal{F})$: outcomes and allowed events; $\mathcal{F}$ closes under complement/countable unions.
- **Measure** (μ): nonnegative, countably additive set size.
- **Random variable**: measurable map $(X: \Omega \rightarrow \mathbb{R})$, making $\{X \leq t\}$ an event.
- **Almost surely**: true except on a probability-zero set; not “always.”

5. Algorithm / Working Process

1. State outcome space.
2. Choose measurable events.
3. Assign probability measure.
4. Define random variables and expectations as integrals.

6. Mathematical Foundation

For disjoint (A_i) , $(\mu \cup A_i) = \sum \mu(A_i)$. Expectation is $(E[X] = \int_{\Omega} X dP)$; with a density it is $(\int x p(x) dx)$. A PDF is density relative to a base measure, not point probability.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(0)
estimate = np.mean(rng.random(1_000_000) ** 2) #  $\int_0^1 x^2 dx$ 
assert abs(estimate - 1/3) < .002
print(estimate)
```

8. Code Explanation

Uniform samples define a probability measure on $([0,1])$; their mean estimates the integral/expectation.

9. Training / Evaluation

No model training. Validate simulation with repeated seeds, confidence intervals, and normalization under the intended base measure.

10. Complexity and Cost

Monte Carlo is $(O(n))$ time, streaming $(O(1))$ memory, and typically $(O(1/\sqrt{n}))$ error. GPUs can vectorize samples.

11. Common Use Cases

Continuous likelihoods, MCMC, Gaussian processes, normalizing flows, and learning-theory proofs.

12. Common Mistakes

Calling every subset an event, confusing density/probability, treating zero probability as impossible, and mixing base measures.

13. Edge Cases / Limitations

It is abstract; densities may not exist; expectations can diverge.

14. Variations

- **Lebesgue integration:** broad integration framework; theoretical foundation.
- **Product measures:** joint distributions; probability essential.
- **Radon–Nikodym derivative:** general density notion; advanced research.

15. Related Topics

Probability spaces formalize random variables; integration formalizes expectation; Bayesian inference uses measures on parameters.

16. Interview Questions

1. **Probability measure?** Countably additive measure totaling one.
2. **Sigma-algebra?** Measurable events closed under set operations.
3. **Why needed?** Well-defined probability/integration.
4. **Almost surely?** Except a zero-probability set.
5. **Always?** No.
6. **PDF versus probability?** Integrate a PDF to get probability.
7. **Expectation formally?** Integral with respect to (P).
8. **Why countable additivity?** Consistency over infinite disjoint events.
9. **Can a zero-probability value occur?** Yes, continuously.
10. **Why measurability?** It makes threshold events valid.

17. Practice Tasks

Check sigma-algebra closure; simulate integrals; compare PMF/PDF; estimate a rare event; explain a.s. convergence.

18. Project Ideas

- **Monte Carlo lab:** NumPy/Matplotlib; analytic integrals; math signal.
- **Distribution simulator:** empirical CDF/PDF comparisons; probability foundations.
- **MCMC estimator:** PyMC/NumPy; small posterior model; research signal.

19. Quick Revision

Key idea: rigorous probability/integration. Formula: $E[X] = \int X dP$. Trap: density is not probability. One-liner: probability is a total-mass-one measure.

20. Final Cheat Sheet

Item	Summary
Definition	Measurable spaces and measures

Item	Summary
Input/output	space, events, measure $\rightarrow$ expectation
Steps	define $(\Omega, \mathcal{F}, P, X)$
Metrics	normalization, Monte Carlo error
Pros/cons	rigorous / abstract
Best uses	probability theory, Bayesian ML

Variational Inference Math

1. Overview

Variational inference (VI) replaces an intractable posterior with a tractable approximation optimized by gradients. It enables VAEs, scalable Bayesian models, topic models, and approximate Gaussian processes.

2. Intuition

Exact Bayes integrates over every hidden explanation. VI chooses a manageable distribution family and finds the member closest to the true posterior.

3. Prerequisites

Bayes rule, KL divergence, expectation, gradients, Monte Carlo sampling, and neural networks for VAEs.

4. Core Concepts

- **Posterior:** $p(z|x) = p(x,z)/p(x)$; evidence is commonly intractable.
- **Variational family:** $q_\phi(z|x)$, often a diagonal Gaussian.
- **ELBO:** optimizable lower bound on evidence; gap is posterior KL.
- **Reparameterization:** $(z = \mu + \sigma \odot \epsilon)$, ($\epsilon \sim N(0, I)$), enables backpropagation.

5. Algorithm / Working Process

1. Specify $p(x,z) = p(x|z)p(z)$.
2. Choose $q_\phi(z|x)$.
3. Reparameterize and sample latent variables.
4. Estimate/maximize ELBO with gradient descent.
5. Use q for inference and decoder likelihood for prediction/generation.

6. Mathematical Foundation

$(ELBO = E_q[\log p(x|z)] - KL(q(z|x)||p(z)))$, and $(\log p(x) = ELBO + KL(q||p(z|x)))$. Mean-field $(q(z) = \prod_i q_i(z_i))$ is fast but misses correlations.

7. Practical Implementation

```
import numpy as np
def gaussian_kl(mu, logvar):
    return .5 * np.sum(np.exp(logvar) + mu**2 - 1 - logvar, axis=-1)
mu = np.array([[.2, -.1]]); logvar = np.array([[0., np.log(.5)]])
kl = gaussian_kl(mu, logvar)
assert kl[0] >= 0
print(kl)
```

8. Code Explanation

This is the closed-form KL from a diagonal Gaussian encoder to a standard-normal prior. Log variance keeps variance positive after exponentiation.

9. Training / Evaluation

Track ELBO, reconstruction term, and KL separately. Evaluate held-out likelihood estimates, sample quality, downstream performance, and calibration. Tune latent size, learning rate, encoder capacity, and KL weight.

10. Complexity and Cost

Each update costs encoder/decoder passes times Monte Carlo samples (often one). Mean-field stores $O(d)$ parameters per example; neural VI normally uses GPUs.

11. Common Use Cases

VAEs, Bayesian neural nets, latent-variable models, topic models, probabilistic embeddings, and approximate posterior prediction.

12. Common Mistakes

Calling ELBO exact likelihood, reversing KL, wrong reconstruction scale, posterior collapse, and evaluating only nice samples.

13. Edge Cases / Limitations

Mean-field underestimates uncertainty; reverse-KL behavior can miss modes; discrete latents need score-function/relaxed estimators.

14. Variations

- **Mean-field VI:** factorized and fast; placement core.
- **Amortized VI:** an encoder predicts variational parameters; VAE essential.
- **Normalizing-flow VI:** richer posterior; research valuable but costlier.

15. Related Topics

Bayesian inference supplies the target; KL measures approximation; VAEs use amortized VI; MCMC is slower but asymptotically exact.

16. Interview Questions

1. **Why VI?** Exact posterior integration is often intractable.
2. **ELBO?** Lower bound on log evidence.
3. **ELBO terms?** Expected log likelihood minus KL to prior.
4. **Why lower bound?** Gap is nonnegative posterior KL.
5. **Amortization?** One inference network serves all examples.
6. **Why reparameterize?** Low-variance differentiable sampling.
7. **VI versus MCMC?** Fast biased optimization versus sampling accuracy.
8. **Posterior collapse?** Latent is ignored and q matches prior.
9. **Beta-VAE?** Reweights KL.
10. **Mean-field weakness?** Missed dependence and narrow uncertainty.

17. Practice Tasks

Derive ELBO; implement Gaussian KL; train MNIST VAE; vary beta; compare VI/MCMC on a 2D posterior.

18. Project Ideas

- **MNIST VAE:** PyTorch/MNIST; generative-AI portfolio.
- **Bayesian regression:** Pyro/PyTorch/UCI; uncertainty bands.
- **Topic explorer:** PyTorch or scikit-learn/20 Newsgroups; NLP research signal.

19. Quick Revision

Key idea: turn inference into optimization. Formula: $(\mathbb{E}_q[\log p(x|z)] - \text{KL}(q||p))$. Trap: ELBO is a bound. One-liner: VI optimizes a tractable posterior approximation.

20. Final Cheat Sheet

Item	Summary
Definition	Optimization-based approximate Bayes
Input/output	model + data $\rightarrow$ q(z)
Steps	choose q, estimate ELBO, optimize
Hyperparameters	latent size, LR, KL weight
Pros/cons	scalable / biased approximation
Best uses	VAEs, Bayesian neural models

Bayesian Nonparametrics

1. Overview

Bayesian nonparametrics uses priors whose effective complexity can grow with data. It is used for unknown-count clustering, density estimation, topic models, and Gaussian-process regression.

2. Intuition

Rather than fixing ten clusters in advance, a model can create another cluster when data justify it while still preferring simple explanations.

3. Prerequisites

Bayes rule, distributions, mixture models, latent variables, exchangeability, and approximate inference.

4. Core Concepts

- **Dirichlet process (DP):** $(G \sim \text{DP}(\alpha, G_0))$, a distribution over distributions centered on base (G_0) .
- **Chinese restaurant process (CRP):** assignment view of the DP: join large clusters or open a new one.
- **Concentration (alpha):** larger values favor more clusters; expected count grows about $(\alpha \log n)$.
- **Gaussian process (GP):** a distribution over functions specified by mean and kernel.

5. Algorithm / Working Process

1. Choose DP mixture or GP prior and hyperparameters.
2. Observe data and infer posterior using MCMC or VI.
3. For DP, probabilistically use existing/new clusters.
4. Predict while integrating posterior uncertainty.

6. Mathematical Foundation

CRP assignment is $(P(z_n=k) = n_k / (n-1 + \alpha))$ for existing cluster and $(P(\text{new}) = \alpha / (n-1 + \alpha))$. For $(f \sim \text{GP}(m, k))$, $(f(X) \sim N(m(X), K(X, X)))$; kernel encodes function similarity.

7. Practical Implementation

```
import numpy as np
def crp(n, alpha=1., seed=0):
    rng, counts = np.random.default_rng(seed), []
    for _ in range(n):
        p = np.array(counts + [alpha], float); i = rng.choice(len(p), p=p/p.sum())
        if i == len(counts): counts.append(0)
        counts[i] += 1
    return counts
counts = crp(100, alpha=2)
assert sum(counts) == 100
print(len(counts), counts)
```

8. Code Explanation

Each observation joins a cluster proportionally to membership or starts one with weight (alpha). This samples a CRP prior; it is not fitted clustering.

9. Training / Evaluation

Use splits respecting groups/time. Assess predictive log likelihood, calibration, cluster stability/ARI when labels exist, and uncertainty. Tune (alpha), base prior, kernel, noise, and inducing points.

10. Complexity and Cost

DP inference depends on sampler/VI. Exact GP training needs ($O(n^3)$) time and ($O(n^2)$) memory; sparse GPs scale farther.

11. Common Use Cases

Customer/entity clustering, topic discovery, anomaly detection, spatial interpolation, Bayesian optimization, and few-shot regression.

12. Common Mistakes

Saying nonparametric means parameter-free, assuming infinite occupied clusters, treating (alpha) as a cluster count, ignoring label switching, and using exact GP on huge data.

13. Edge Cases / Limitations

Inference is slow and prior-sensitive; DP mixtures can make tiny clusters; bad kernels give misleading GP uncertainty.

14. Variations

- **DP mixture:** unknown-number clustering; research/interview important.
- **Hierarchical DP:** shares topics across groups; topic-model use.
- **Sparse GP:** inducing points reduce cost; practical large-data variant.

15. Related Topics

Finite Gaussian mixtures fix component counts; VI/MCMC infer process posteriors; kernels link GPs to Bayesian function priors.

16. Interview Questions

1. **Nonparametric means?** Effective complexity may grow with data.
2. **Parameter-free?** No.
3. **What does alpha do?** Controls new-cluster tendency.
4. **CRP?** Exchangeable prior on partitions.
5. **Infinite active clusters?** Finite occupied for finite data.
6. **GP?** Distribution over functions.
7. **Kernel role?** Covariance/similarity assumption.
8. **Why GP cubic?** Dense kernel factorization.
9. **DP mixture versus k-means?** Probabilistic unknown count versus fixed hard clusters.
10. **GP scaling?** Sparse inducing-point approximations.

17. Practice Tasks

Simulate CRPs under several alphas; fit mixture models; train GP regression; compare exact/sparse GPs; inspect uncertainty.

18. Project Ideas

- **Customer segments:** PyMC/scikit-learn/Mall Customers; Bayesian clustering.
- **Air-quality interpolation:** GPyTorch/sensor data; uncertainty-aware regression.
- **Topic discovery:** Gensim/Pyro/news corpus; NLP research value.

19. Quick Revision

Key idea: data-adaptive model complexity. Formula: $P(\text{new}) = \alpha / (n - 1 + \alpha)$. Trap: not parameter-free. One-liner: DP priors allow clusters without preselecting count.

20. Final Cheat Sheet

Item	Summary
Definition	Bayesian models with adaptable complexity
Input/output	data + process prior → posterior structure
Steps	choose process, infer, predict
Hyperparameters	alpha, base prior, kernel/noise
Pros/cons	flexible uncertainty / costly inference
Best uses	unknown clusters, GPs, topics